Chapter Ten
PID Control

Based on a survey of over eleven thousand controllers in the refining, chemicals and pulp and paper industries, 97% of regulatory controllers utilize PID feedback.

L. Desborough and R. Miller, 2002 [DM02].

This chapter treats the basic properties of proportional-integral-derivative (PID) control and the methods for choosing the parameters of the controllers. We also analyze the effects of actuator saturation and time delay, two important features of many feedback systems, and describe methods for compensating for these effects. Finally, we will discuss the implementation of PID controllers as an example of how to implement feedback control systems using analog or digital computation.

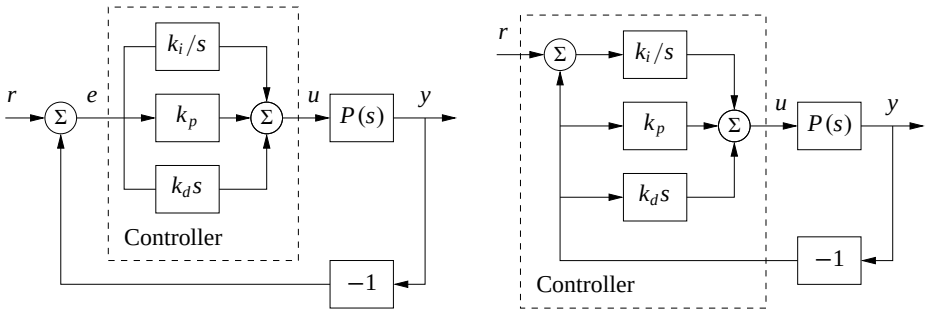
10.1 Basic Control Functions

PID control, which was introduced in Section 1.5 and has been used in several examples, is by far the most common way of using feedback in engineering systems. It appears in simple devices and in large factories with thousands of controllers. PID controllers appear in many different forms: as stand-alone controllers, as part of hierarchical, distributed control systems and built into embedded components. Most PID controllers do not use derivative action, so they should strictly speaking be called PI controllers; we will, however, use PID as a generic term for this class of controller. There is also growing evidence that PID control appears in biological systems [YHSD00].

Block diagrams of closed loop systems with PID controllers are shown in Figure 10.1. The control signal u for the system in Figure 10.1a is formed entirely from the error e; there is no feedforward term (which would correspond to $k_r r$ in the state feedback case). A common alternative in which proportional and derivative action do not act on the reference is shown in Figure 10.1b; combinations of the schemes will be discussed in Section 10.5. The command signal r is called the reference signal in regulation problems, or the *setpoint* in the literature of PID control. The input/output relation for an ideal PID controller with error feedback is

$$u = k_p e + k_i \int_0^t e(\tau)\,d\tau + k_d \frac{de}{dt} = k_p\Big(e + \frac{1}{T_i}\int_0^t e(\tau)\,d\tau + T_d\frac{de}{dt}\Big). \quad (10.1)$$

The control action is thus the sum of three terms: proportional feedback, the integral term and derivative action. For this reason PID controllers were originally called *three-term controllers*. The controller parameters are the proportional gain k_p, the



(a) PID using error feedback (b) PID using two degrees of freedom

Figure 10.1: Block diagrams of closed loop systems with ideal PID controllers. Both controllers have one output, the control signal u. The controller in (a), which is based on error feedback, has one input, the control error $e = r - y$. For this controller proportional, integral and derivative action acts on the error $e = r - y$. The two degree-of-freedom controller in (b) has two inputs, the reference r and the process output y. Integral action acts on the error, but proportional and derivative action act on the process output y.

integral gain k_i and the derivative gain k_d. The time constants T_i and T_d, called integral time (constant) and derivative time (constant), are sometimes used instead of the integral and derivative gains.

The controller (10.1) represents an idealized controller. It is a useful abstraction for understanding the PID controller, but several modifications must be made to obtain a controller that is practically useful. Before discussing these practical issues we will develop some intuition about PID control.

We start by considering pure proportional feedback. Figure 10.2a shows the responses of the process output to a unit step in the reference value for a system with pure proportional control at different gain settings. In the absence of a feedforward term, the output never reaches the reference, and hence we are left with nonzero steady-state error. Letting the process and the controller have transfer functions $P(s)$ and $C(s)$, the transfer function from reference to output is

$$G_{yr} = \frac{PC}{1 + PC}, \tag{10.2}$$

and thus the steady-state error for a unit step is

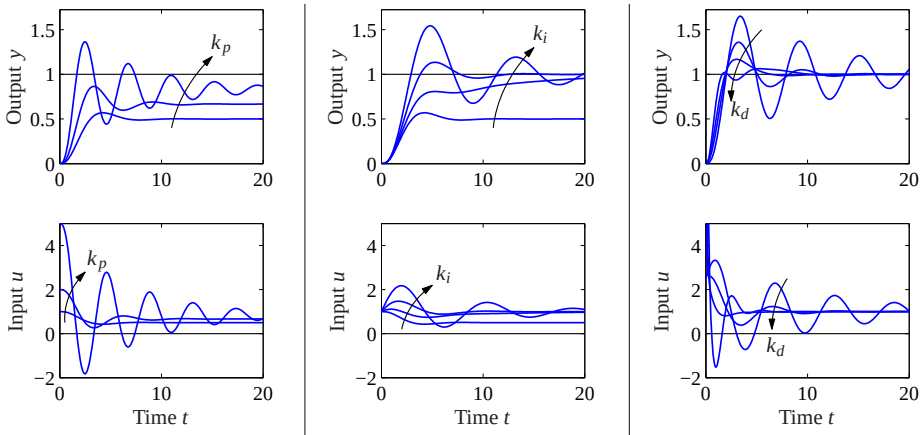
$$1 - G_{yr}(0) = \frac{1}{1 + k_p P(0)}.$$

For the system in Figure 10.2a with gains $k_p = 1$, 2 and 5, the steady-state error is 0.5, 0.33 and 0.17. The error decreases with increasing gain, but the system also becomes more oscillatory. Notice in the figure that the initial value of the control signal equals the controller gain.

To avoid having a steady-state error, the proportional term can be changed to

$$u(t) = k_p e(t) + u_{\text{ff}}, \tag{10.3}$$

where u_{ff} is a feedforward term that is adjusted to give the desired steady-state



(a) Proportional control (b) PI control (c) PID control

Figure 10.2: Responses to step changes in the reference value for a system with a proportional controller (a), PI controller (b) and PID controller (c). The process has the transfer function $P(s) = 1/(s+1)^3$, the proportional controller has parameters $k_p = 1$, 2 and 5, the PI controller has parameters $k_p = 1$, $k_i = 0$, 0.2, 0.5 and 1, and the PID controller has parameters $k_p = 2.5$, $k_i = 1.5$ and $k_d = 0$, 1, 2 and 4.

value. If we choose $u_{\text{ff}} = r/P(0) = k_r r$, then the output will be exactly equal to the reference value, as it was in the state space case, provided that there are no disturbances. However, this requires exact knowledge of the process dynamics, which is usually not available. The parameter u_{ff}, called *reset* in the PID literature, must therefore be adjusted manually.

As we saw in Section 6.4, integral action guarantees that the process output agrees with the reference in steady state and provides an alternative to the feedforward term. Since this result is so important, we will provide a general proof. Consider the controller given by equation (10.1). Assume that there exists a steady state with $u = u_0$ and $e = e_0$. It then follows from equation (10.1) that

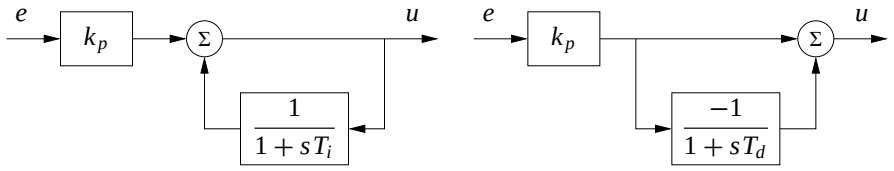
$$u_0 = k_p e_0 + k_i e_0 t,$$

which is a contradiction unless e_0 or k_i is zero. We can thus conclude that with integral action the error will be zero if it reaches a steady state. Notice that we have not made any assumptions about the linearity of the process or the disturbances. We have, however assumed that an equilibrium exists. Using integral action to achieve zero steady-state error is much better than using feedforward, which requires a precise knowledge of process parameters.

The effect of integral action can also be understood from frequency domain analysis. The transfer function of the PID controller is

$$C(s) = k_p + \frac{k_i}{s} + k_d s. \tag{10.4}$$

The controller has infinite gain at zero frequency ($C(0) = \infty$), and it then follows from equation (10.2) that $G_{yr}(0) = 1$, which implies that there is no steady-state



(a) Automatic reset (b) Proportional-derivative

Figure 10.3: Implementation of PI and PD controllers. The block diagram in (a) shows how integral action is implemented using *positive feedback* with a first-order system, sometimes called automatic reset. The block diagram in (b) shows how derivative action can be implemented by taking differences between a static system and a first-order system.

error for a step input.

Integral action can also be viewed as a method for generating the feedforward term u_{ff} in the proportional controller (10.3) automatically. One way to do this is shown in Figure 10.3a, where the controller output is low-pass-filtered and fed back with positive gain. This implementation, called *automatic reset*, was one of the early inventions of integral control. The transfer function of the system in Figure 10.3a is obtained by block diagram algebra; we have

$$G_{ue} = k_p \frac{1 + sT_i}{sT_i} = k_p + \frac{k_p}{sT_i},$$

which is the transfer function for a PI controller.

The properties of integral action are illustrated in Figure 10.2b for a step input. The proportional gain is constant, $k_p = 1$, and the integral gains are $k_i = 0$, 0.2, 0.5 and 1. The case $k_i = 0$ corresponds to pure proportional control, with a steady-state error of 50%. The steady-state error is eliminated when integral gain action is used. The response creeps slowly toward the reference for small values of k_i and goes faster for larger integral gains, but the system also becomes more oscillatory.

The integral gain k_i is a useful measure for attenuation of load disturbances. Consider a closed loop system under PID control and assume that the system is stable and initially at rest with all signals being zero. Apply a unit step disturbance at the process input. After a transient the process output goes to zero and the controller output settles at a value that compensates for the disturbance. It follows from (10.1) that

$$u(\infty) = k_i \int_0^\infty e(t)dt.$$

The integrated error is thus inversely proportional to the integral gain k_i. The integral gain is thus a measure of the effectiveness of disturbance attenuation. A large gain k_i attenuates disturbances effectively, but too large a gain gives oscillatory behavior, poor robustness and possibly instability.

We now return to the general PID controller and consider the effect of the derivative term k_d. Recall that the original motivation for derivative feedback was to provide predictive or anticipatory action. Notice that the combination of the

proportional and the derivative terms can be written as

$$u = k_p e + k_d \frac{de}{dt} = k_p\big(e + T_d \frac{de}{dt}\big) = k_p e_p,$$

where $e_p(t)$ can be interpreted as a prediction of the error at time $t + T_d$ by linear extrapolation. The prediction time $T_d = k_d/k_p$ is the derivative time constant of the controller.

Derivative action can be implemented by taking the difference between the signal and its low-pass filtered version as shown in Figure 10.3b. The transfer function for the system is

$$G_{ue}(s) = k_p\Big(1 - \frac{1}{1 + sT_d}\Big) = k_p \frac{sT_d}{1 + sT_d}. \tag{10.5}$$

The system thus has the transfer function $G(s) = sT_d/(1 + sT_d)$, which approximates a derivative for low frequencies ($|s| < T_d$).

Figure 10.2c illustrates the effect of derivative action: the system is oscillatory when no derivative action is used, and it becomes more damped as the derivative gain is increased. Performance deteriorates if the derivative gain is too high. When the input is a step, the controller output generated by the derivative term will be an impulse. This is clearly visible in Figure 10.2c. The impulse can be avoided by using the controller configuration shown in Figure 10.1b.

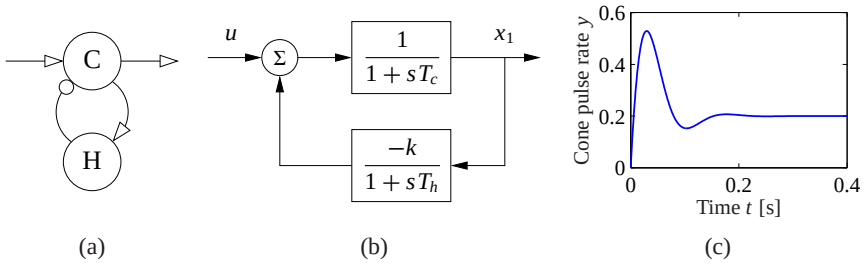
Although PID control was developed in the context of engineering applications, it also appears in nature. Disturbance attenuation by feedback in biological systems is often called *adaptation*. A typical example is the pupillary reflex discussed in Example 8.11, where it is said that the eye adapts to changing light intensity. Analogously, feedback with integral action is called perfect adaptation [YHSD00]. In biological systems proportional, integral and derivative action is generated by combining subsystems with dynamical behavior similarly to what is done in engineering systems. For example, PI action can be generated by the interaction of several hormones [ESGK02].

Example 10.1 PD action in the retina

The response of cone photoreceptors in the retina is an example where proportional and derivative action is generated by a combination of cones and horizontal cells. The cones are the primary receptors stimulated by light, which in turn stimulate the horizontal cells, and the horizontal cells give inhibitory (negative) feedback to the cones. A schematic diagram of the system is shown in Figure 10.4a. The system can be modeled by ordinary differential equations by representing neuron signals as continuous variables representing the average pulse rate. In [Wil99] it is shown that the system can be represented by the differential equations

$$\frac{dx_1}{dt} = \frac{1}{T_c}(-x_1 - kx_2 + u), \qquad \frac{dx_2}{dt} = \frac{1}{T_h}(x_1 - x_2),$$

where u is the light intensity and x_1 and x_2 are the average pulse rates from the cones and the horizontal cells. A block diagram of the system is shown in Figure 10.4b.



(a) (b) (c)

Figure 10.4: Schematic diagram of cone photoreceptors (C) and horizontal cells (H) in the retina. In the schematic diagram in (a), excitatory feedback is indicated by arrows and inhibitory feedback by circles. A block diagram is shown in (b) and the step response in (c).

The step response of the system shown in Figure 10.4c shows that the system has a large initial response followed by a lower, constant steady-state response typical of proportional and derivative action. The parameters used in the simulation are $k = 4$, $T_c = 0.025$ and $T_h = 0.08$. ∇

10.2 Simple Controllers for Complex Systems

Many of the design methods discussed in previous chapters have the property that the complexity of the controller is directly reflected by the complexity of the model. When designing controllers by output feedback in Chapter 7, we found for single-input, single-output systems that the order of the controller was the same as the order of the model, possibly one order higher if integral action was required. Applying similar design methods for PID control will require that we have low-order models of the processes to be able to easily analyze the results.

Low-order models can be obtained from first principles. Any stable system can be modeled by a static system if its inputs are sufficiently slow. Similarly a first-order model is sufficient if the storage of mass, momentum or energy can be captured by only one variable; typical examples are the velocity of a car on a road, angular velocity of a stiff rotational system, the level in a tank and the concentration in a volume with good mixing. System dynamics are of second order if the storage of mass, energy and momentum can be captured by two state variables; typical examples are the position of a car on the road, the stabilization of stiff satellites, the levels in two connected tanks and two-compartment models. A wide range of techniques for model reduction are also available. In this chapter we will focus on design techniques where we simplify the models to capture the essential properties that are needed for PID design.

We begin by analyzing the case of integral control. A stable system can be controlled by an integral controller provided that the requirements on the closed loop system are modest. To design the controller we assume that the transfer function of the process is a constant $K = P(0)$. The loop transfer function under integral control then becomes Kk_i/s, and the closed loop characteristic polynomial is simply

$s + Kk_i$. Specifying performance by the desired time constant T_{cl} of the closed loop system, we find that the integral gain is given by

$$k_i = 1/(T_{cl}P(0)).$$

The analysis requires that T_{cl} be sufficiently large that the process transfer function can be approximated by a constant.

For systems that are not well represented by a constant gain, we can obtain a better approximation by using the Taylor series expansion of the loop transfer function:

$$L(s) = \frac{k_i P(s)}{s} \approx \frac{k_i(P(0) + sP'(0))}{s} = k_i P'(0) + \frac{k_i P(0)}{s}.$$

Choosing $k_i P'(0) = -0.5$ gives a system with good robustness, as will be discussed in Section 12.5. The controller gain is then given by

$$k_i = -\frac{1}{2P'(0)}, \tag{10.6}$$

and the expected closed loop time constant is $T_{cl} \approx -2P'(0)/P(0)$.

Example 10.2 Integral control of AFM in tapping mode
A simplified model of the dynamics of the vertical motion of an atomic force microscope in tapping mode was discussed in Exercise 9.2. The transfer function for the system dynamics is

$$P(s) = \frac{a(1 - e^{-s\tau})}{s\tau(s + a)},$$

where $a = \zeta\omega_0$, $\tau = 2\pi n/\omega_0$ and the gain has been normalized to 1. We have $P(0) = 1$ and $P'(0) = -\tau/2 - 1/a$, and it follows from (10.6) that the integral gain can be chosen as $k_i = a/(2 + a\tau)$. Nyquist and Bode plots for the resulting loop transfer function are shown in Figure 10.5. ∇

A first-order system has the transfer function

$$P(s) = \frac{b}{s + a}.$$

With a PI controller the closed loop system has the characteristic polynomial

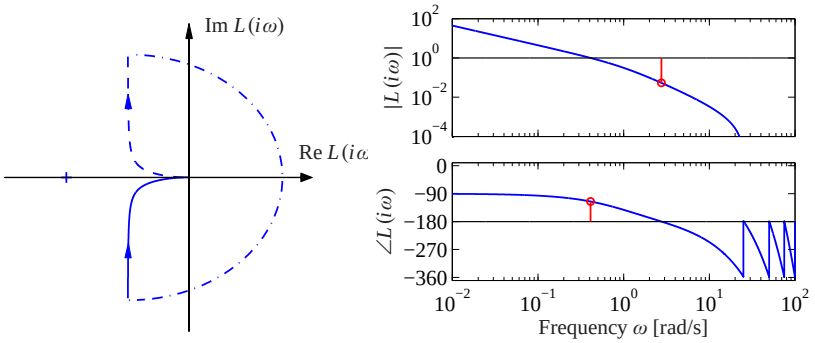
$$s(s + a) + bk_p s + bk_i s = s^2 + (a + bk_p)s + bk_i.$$

The closed loop poles can thus be assigned arbitrary values by proper choice of the controller gains. Requiring that the closed loop system have the characteristic polynomial

$$p(s) = s^2 + a_1 s + a_2^2,$$

we find that the controller parameters are

$$k_p = \frac{a_1 - a}{b}, \qquad k_i = \frac{a_2}{b}. \tag{10.7}$$



(a) Nyquist plot (b) Bode plot

Figure 10.5: Integral control for AFM in tapping mode. An integral controller is designed based on the slope of the process transfer function at 0. The controller gives good robustness properties based on a very simple analysis.

If we require a response of the closed loop system that is slower than that of the open loop system, a reasonable choice is $a_1 = a + \alpha$ and $a_2 = \alpha a$. If a response faster than that of the open loop system is required, it is reasonable to choose $a_1 = 2\zeta\omega_0$ and $a_2 = \omega_0^2$, where ω_0 and ζ are undamped natural frequency and damping ratio of the dominant mode. These choices have significant impact on the robustness of the system and will be discussed in Section 12.4. An upper limit to ω_0 is given by the validity of the model. Large values of ω_0 will require fast control actions, and actuators may saturate if the value is too large. A first-order model is unlikely to represent the true dynamics for high frequencies. We illustrate the design by an example.

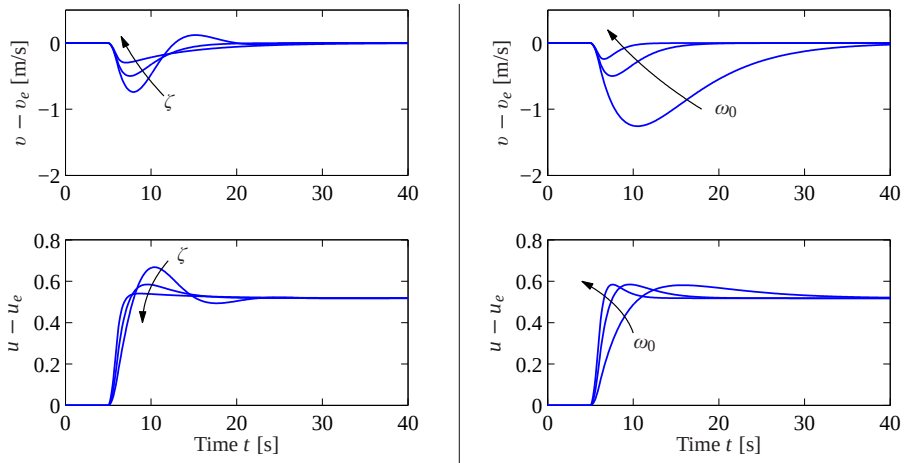
Example 10.3 Cruise control using PI feedback

Consider the problem of maintaining the speed of a car as it goes up a hill. In Example 5.14 we found that there was little difference between the linear and nonlinear models when investigating PI control, provided that the throttle did not reach the saturation limits. A simple linear model of a car was given in Example 5.11:

$$\frac{d(v - v_e)}{dt} = -a(v - v_e) + b(u - u_e) - g\theta, \tag{10.8}$$

where v is the velocity of the car, u is the input from the engine and θ is the slope of the hill. The parameters were $a = 0.0101$, $b = 1.3203$, $g = 9.8$, $v_e = 20$ and $u_e = 0.1616$. This model will be used to find suitable parameters of a vehicle speed controller. The transfer function from throttle to velocity is a first-order system. Since the open loop dynamics is so slow, it is natural to specify a faster closed loop system by requiring that the closed loop system be of second-order with damping ratio ζ and undamped natural frequency ω_0. The controller gains are given by (10.7).

Figure 10.6 shows the velocity and the throttle for a car that initially moves on a horizontal road and encounters a hill with a slope of 4° at time $t = 6$ s. To design a PI controller we choose $\zeta = 1$ to obtain a response without overshoot, as



(a) $\omega_0 = 0.5$, $\zeta = 0.5, 1, 2$ (b) $\zeta = 1$, ω_0 = 0.2, 0.5, 1

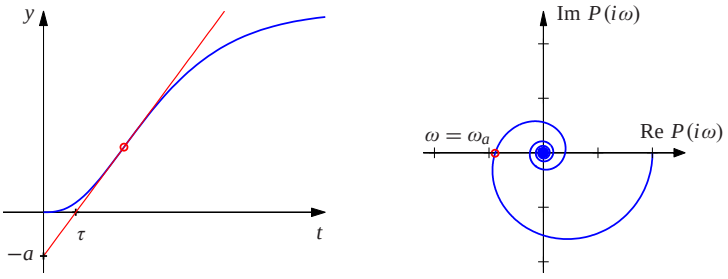
Figure 10.6: Cruise control using PI feedback. The step responses for the error and input illustrate the effect of parameters $\zeta = 1$ and ω_0 on the response of a car with cruise control. A change in road slope from 0° to 4° is applied between $t = 5$ and 6 s. (a) Responses for $\omega_0 = 0.5$ and $\zeta = 0.5$, 1 and 2. Choosing $\zeta = 1$ gives no overshoot. (b) Responses for $\zeta = 1$ and $\omega_0 = 0.2$, 0.5 and 1.0.

shown in Figure 10.6a. The choice of ω_0 is a compromise between response speed and control actions: a large value gives a fast response, but it requires fast control action. The trade-off is is illustrated in Figure 10.6b. The largest velocity error decreases with increasing ω_0, but the control signal also changes more rapidly. In the simple model (10.8) it was assumed that the force responds instantaneously to throttle commands. For rapid changes there may be additional dynamics that have to be accounted for. There are also physical limitations to the rate of change of the force, which also restricts the admissible value of ω_0. A reasonable choice of ω_0 is in the range 0.5–1.0. Notice in Figure 10.6 that even with $\omega_0 = 0.2$ the largest velocity error is only 1 m/s. ∇

A PI controller can also be used for a process with second-order dynamics, but there will be restrictions on the possible locations of the closed loop poles. Using a PID controller, it is possible to control a system of second order in such a way that the closed loop poles have arbitrary locations; see Exercise 10.2.

Instead of finding a low-order model and designing controllers for them, we can also use a high-order model and attempt to place only a few dominant poles. An integral controller has one parameter, and it is possible to position one pole. Consider a process with the transfer function $P(s)$. The loop transfer function with an integral controller is $L(s) = k_i P(s)/s$. The roots of the closed loop characteristic polynomial are the roots of $s + k_i P(s) = 0$. Requiring that $s = -a$ be a root, we find that the controller gain should be chosen as

$$k_i = \frac{a}{P(-a)}. \tag{10.9}$$



(a) Step response method (b) Frequency response method

Figure 10.7: Ziegler–Nichols step and frequency response experiments. The unit step response in (a) is characterized by the parameters a and τ. The frequency response method (b) characterizes process dynamics by the point where the Nyquist curve of the process transfer function first intersects the negative real axis and the frequency ω_c where this occurs.

The pole $s = -a$ will be dominant if a is small. A similar approach can be applied to PI and PID controllers.

10.3 PID Tuning

Users of control systems are frequently faced with the task of adjusting the controller parameters to obtain a desired behavior. There are many different ways to do this. One approach is to go through the conventional steps of modeling and control design as described in the previous section. Since the PID controller has so few parameters, a number of special empirical methods have also been developed for direct adjustment of the controller parameters. The first tuning rules were developed by Ziegler and Nichols [ZN42]. Their idea was to perform a simple experiment, extract some features of process dynamics from the experiment and determine the controller parameters from the features.

Ziegler–Nichols' Tuning

In the 1940s, Ziegler and Nichols developed two methods for controller tuning based on simple characterization of process dynamics in the time and frequency domains.

The time domain method is based on a measurement of part of the open loop unit step response of the process, as shown in Figure 10.7a. The step response is measured by applying a unit step input to the process and recording the response. The response is characterized by parameters a and τ, which are the intercepts of the steepest tangent of the step response with the coordinate axes. The parameter τ is an approximation of the time delay of the system and a/τ is the steepest slope of the step response. Notice that it is not necessary to wait until steady state is reached to find the parameters, it suffices to wait until the response has had an inflection point. The controller parameters are given in Table 10.1. The parameters were obtained

Table 10.1: Ziegler–Nichols tuning rules. (a) The step response methods give the parameters in terms of the intercept a and the apparent time delay τ. (b) The frequency response method gives controller parameters in terms of *critical gain* k_c and *critical period* T_c.

Type	k_p	T_i	T_d
P	1/a		
PI	0.9/a	3τ	
PID	1.2/a	2τ	0.5τ

(a) Step response method

Type	k_p	T_i	T_d
P	$0.5k_c$		
PI	$0.4k_c$	$0.8T_c$	
PID	$0.6k_c$	$0.5T_c$	$0.125T_c$

(b) Frequency response method

by extensive simulation of a range of representative processes. A controller was tuned manually for each process, and an attempt was then made to correlate the controller parameters with a and τ.

In the frequency domain method, a controller is connected to the process, the integral and derivative gains are set to zero and the proportional gain is increased until the system starts to oscillate. The critical value of the proportional gain k_c is observed together with the period of oscillation T_c. It follows from Nyquist's stability criterion that the loop transfer function $L = k_c P(s)$ intersects the critical point at the frequency $\omega_c = 2\pi/T_c$. The experiment thus gives the point on the Nyquist curve of the process transfer function where the phase lag is 180°, as shown in Figure 10.7b.

The Ziegler–Nichols methods had a huge impact when they were introduced in the 1940s. The rules were simple to use and gave initial conditions for manual tuning. The ideas were adopted by manufacturers of controllers for routine use. The Ziegler–Nichols tuning rules unfortunately have two severe drawbacks: too little process information is used, and the closed loop systems that are obtained lack robustness.

The step response method can be improved significantly by characterizing the unit step response by parameters K, τ and T in the model

$$P(s) = \frac{K}{1+sT} e^{-\tau s}. \tag{10.10}$$

The parameters can be obtained by fitting the model to a measured step response. Notice that the experiment takes a longer time than the experiment in Figure 10.7a because to determine K it is necessary to wait until the steady state has been reached. Also notice that the intercept a in the Ziegler–Nichols rule is given by $a = K\tau/T$.

The frequency response method can be improved by measuring more points on the Nyquist curve, e.g., the zero frequency gain K or the point where the process has a 90° phase lag. This latter point can be obtained by connecting an integral controller and increasing its gain until the system reaches the stability limit. The experiment can also be automated by using relay feedback, as will be discussed later in this section.

There are many versions of improved tuning rules. As an illustration we give

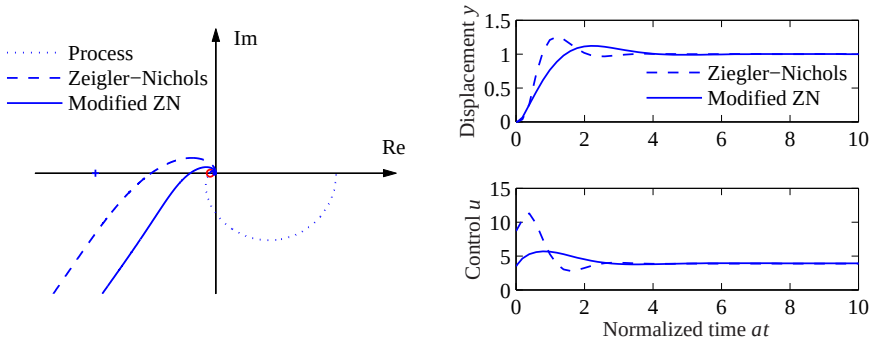


Figure 10.8: PI control of an AFM in tapping mode. Nyquist plots (a) and step responses (b) for PI control of the vertical motion of an atomic force microscope in tapping mode. The averaging parameter is $n = 20$. Results with Ziegler–Nichols tuning are shown by dashed lines, and modified Ziegler–Nichols tuning is shown by solid lines. The Nyquist plot of the process transfer function is shown by dotted lines.

the following rules for PI control, based on [ÅH05]:

$$k_p = \frac{0.15\tau + 0.35T}{K\tau} \quad \left(\frac{0.9T}{K\tau}\right), \quad k_i = \frac{0.46\tau + 0.02T}{K\tau^2} \quad \left(\frac{0.3T}{K\tau^2}\right),$$
$$k_p = 0.22k_c - \frac{0.07}{K} \quad \left(0.4k_c\right), \quad k_i = \frac{0.16k_c}{T_c} + \frac{0.62}{KT_c} \quad \left(\frac{0.5k_c}{T_c}\right). \tag{10.11}$$

The values for the Ziegler–Nichols rule are given in parentheses. Notice that the improved formulas typically give lower controller gains than the Ziegler–Nichols method. The integral gain is higher for systems where the dynamics are delay-dominated, $\tau \gg T$.

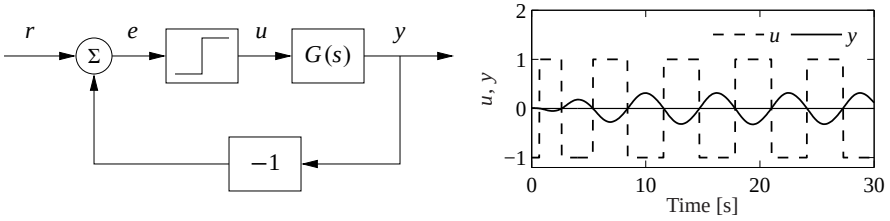
Example 10.4 Atomic force microscope in tapping mode

A simplified model of the dynamics of the vertical motion of an atomic force microscope in tapping mode was discussed in Example 10.2. The transfer function is normalized by choosing $1/a$ as the time unit. The normalized transfer function is

$$P(s) = \frac{1 - e^{-sT_n}}{sT_n(s+1)},$$

where $T_n = 2n\pi a/\omega_0 = 2n\pi\zeta$. The Nyquist plot of the transfer function is shown in Figure 10.8a for $z = 0.002$ and $n = 20$. The leftmost intersection of the Nyquist curve with the real axis occurs at $\mathrm{Re}\, s = -0.0461$ for $\omega = 13.1$. The critical gain is thus $k_c = 21.7$ and the critical period is $T_c = 0.48$. Using the Ziegler–Nichols tuning rule, we find the parameters $k_p = 8.87$ and $k_i = 22.6$ ($T_i = 0.384$) for a PI controller. With this controller the stability margin is $s_m = 0.31$, which is quite small. The step response of the controller is shown in Figure 10.8. Notice in particular that there is a large overshoot in the control signal.

The modified Ziegler–Nichols rule (10.11) gives the controller parameters $k =$



(a) Relay feedback (b) Oscillatory response

Figure 10.9: Block diagram of a process with relay feedback (a) and typical signals (b). The process output y is a solid line, and the relay output u is a dashed line. Notice that the signals u and y have opposite phases.

3.47 and $k_i = 8.73$ ($T_i = 0.459$) and the stability margin becomes $s_m = 0.61$. The step response with this controller is shown in Figure 10.8. A comparison of the responses obtained with the original Ziegler–Nichols rule shows that the overshoot has been reduced. Notice that the control signal reaches its steady-state value almost instantaneously. It follows from Example 10.2 that a pure integral controller has the normalized gain $k_i = 1/(2 + T_n) = 0.44$. Comparing this with the gains of a PI controller, we can conclude that a PI controller gives much better performance than a pure integral controller. ∇

Relay Feedback

The Ziegler–Nichols frequency response method increases the gain of a proportional controller until oscillation to determine the critical gain k_c and the corresponding critical period T_c or, equivalently, the point where the Nyquist curve intersects the negative real axis. One way to obtain this information automatically is to connect the process in a feedback loop with a nonlinear element having a relay function as shown in Figure 10.9a. For many systems there will then be an oscillation, as shown in Figure 10.9b, where the relay output u is a square wave and the process output y is close to a sinusoid. Moreover the input and the output are out of phase, which means that the system oscillates with the critical period T_c, where the process has a phase lag of 180°. Notice that an oscillation with constant period is established quickly.

The critical period is simply the period of the oscillation. To determine the critical gain we expand the square wave relay output in a Fourier series. Notice in the figure that the process output is practically sinusoidal because the process attenuates higher harmonics effectively. It is then sufficient to consider only the first harmonic component of the input. Letting d be the relay amplitude, the first harmonic of the square wave input has amplitude $4d/\pi$. If a is the amplitude of the process output, the process gain at the critical frequency $\omega_c = 2\pi/T_c$ is $|P(i\omega_c)| = \pi a/(4d)$ and the critical gain is

$$K_c = \frac{4d}{a\pi}. \tag{10.12}$$

Having obtained the critical gain K_c and the critical period T_c, the controller parameters can then be determined using the Ziegler–Nichols rules. Improved tuning can be obtained by fitting a model to the data obtained from the relay experiment.

The relay experiment can be automated. Since the amplitude of the oscillation is proportional to the relay output, it is easy to control it by adjusting the relay output. *Automatic tuning* based on relay feedback is used in many commercial PID controllers. Tuning is accomplished simply by pushing a button that activates relay feedback. The relay amplitude is automatically adjusted to keep the oscillations sufficiently small, and the relay feedback is switched to a PID controller as soon as the tuning is finished.

10.4 Integrator Windup

Many aspects of a control system can be understood from linear models. There are, however, some nonlinear phenomena that must be taken into account. These are typically limitations in the actuators: a motor has limited speed, a valve cannot be more than fully opened or fully closed, etc. For a system that operates over a wide range of conditions, it may happen that the control variable reaches the actuator limits. When this happens, the feedback loop is broken and the system runs in open loop because the actuator remains at its limit independently of the process output as long as the actuator remains saturated. The integral term will also build up since the error is typically nonzero. The integral term and the controller output may then become very large. The control signal will then remain saturated even when the error changes, and it may take a long time before the integrator and the controller output come inside the saturation range. The consequence is that there are large transients. This situation is referred to as *integrator windup*, illustrated in the following example.

Example 10.5 Cruise control
The windup effect is illustrated in Figure 10.10a, which shows what happens when a car encounters a hill that is so steep (6°) that the throttle saturates when the cruise controller attempts to maintain speed. When encountering the slope at time $t = 5$, the velocity decreases and the throttle increases to generate more torque. However, the torque required is so large that the throttle saturates. The error decreases slowly because the torque generated by the engine is just a little larger than the torque required to compensate for gravity. The error is large and the integral continues to build up until the error reaches zero at time 30, but the controller output is still larger than the saturation limit and the actuator remains saturated. The integral term starts to decrease, and at time 45 and the velocity settles quickly to the desired value. Notice that it takes considerable time before the controller output comes into the range where it does not saturate, resulting in a large overshoot. ∇

There are many methods to avoid windup. One method is illustrated in Figure 10.11: the system has an extra feedback path that is generated by measuring the actual actuator output, or the output of a mathematical model of the saturating

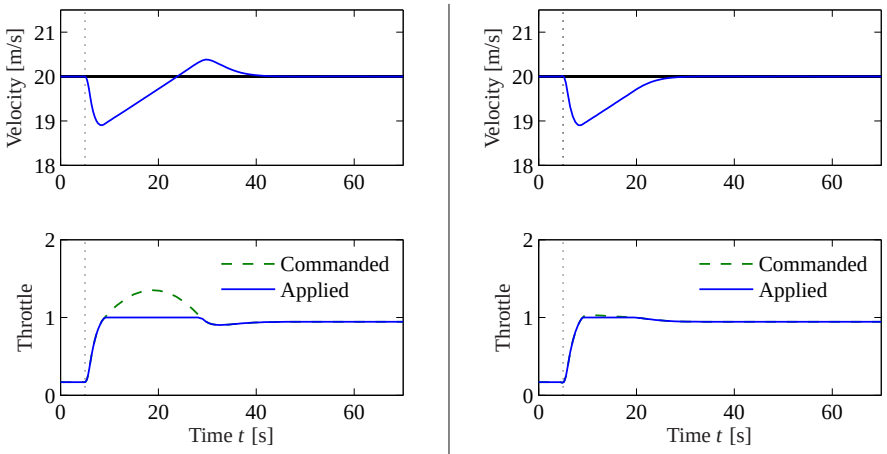


Figure 10.10: Simulation of PI cruise control with windup (a) and anti-windup (b). The figure shows the speed v and the throttle u for a car that encounters a slope that is so steep that the throttle saturates. The controller output is a dashed line. The controller parameters are $k_p = 0.5$ and $k_i = 0.1$. The anti-windup compensator eliminates the overshoot by preventing the error for building up in the integral term of the controller.

actuator, and forming an error signal e_s as the difference between the output of the controller v and the actuator output u. The signal e_s is fed to the input of the integrator through gain k_t. The signal e_s is zero when there is no saturation and the extra feedback loop has no effect on the system. When the actuator saturates, the signal e_s is fed back to the integrator in such a way that e_s goes toward zero. This implies that controller output is kept close to the saturation limit. The controller output will then change as soon as the error changes sign and integral windup is avoided.

The rate at which the controller output is reset is governed by the feedback gain k_t; a large value of k_t gives a short reset time. The parameter k_t cannot be too large because measurement noise can then cause an undesirable reset. A reasonable choice is to choose k_i as a fraction of $1/T_i$. We illustrate how integral windup can be avoided by investigating the cruise control system.

Example 10.6 Cruise control with anti-windup

Figure 10.10b shows what happens when a controller with anti-windup is applied to the system simulated in Figure 10.10a. Because of the feedback from the actuator model, the output of the integrator is quickly reset to a value such that the controller output is at the saturation limit. The behavior is drastically different from that in Figure 10.10a and the large overshoot is avoided. The tracking gain is $k_t = 2$ in the simulation. ∇

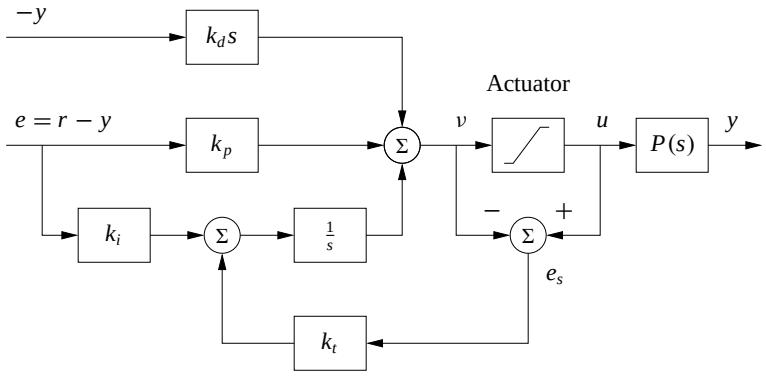


Figure 10.11: PID controller with a filtered derivative and anti-windup. The input to the integrator $(1/s)$ consists of the error term plus a "reset" based on input saturation. If the actuator is not saturated, then $e_s = u - v$, otherwise e_s will decrease the integrator input to prevent windup.

10.5 Implementation

There are many practical issues that have to be considered when implementing PID controllers. They have been developed over time based on practical experience. In this section we consider some of the most common. Similar considerations also apply to other types of controllers.

Filtering the Derivative

A drawback with derivative action is that an ideal derivative has high gain for high-frequency signals. This means that high-frequency measurement noise will generate large variations in the control signal. The effect of measurement noise may be reduced by replacing the term $k_d s$ by $k_d s/(1 + sT_f)$, which can be interpreted as an ideal derivative of a low-pass filtered signal. For small s the transfer function is approximately $k_d s$ and for large s it is equal to k_d/T_f. The approximation acts as a derivative for low-frequency signals and as a constant gain for high-frequency signals. The filtering time is chosen as $T_f = (k_d/k)/N$, with N in the range 2–20. Filtering is obtained automatically if the derivative is implemented by taking the difference between the signal and its filtered version as shown in Figure 10.3b (see equation (10.5)).

Instead of filtering just the derivative, it is also possible to use an ideal controller and filter the measured signal. The transfer function of such a controller with a filter is then

$$C(s) = k_p \left(1 + \frac{1}{sT_i} + sT_d\right) \frac{1}{1 + sT_f + (sT_f)^2/2}, \tag{10.13}$$

where a second-order filter is used.

Setpoint Weighting

Figure 10.1 shows two configurations of a PID controller. The system in Figure 10.1a has a controller with *error feedback* where proportional, integral and derivative action acts on the error. In the simulation of PID controllers in Figure 10.2c there is a large initial peak in the control signal, which is caused by the derivative of the reference signal. The peak can be avoided by using the controller in Figure 10.1b, where proportional and derivative action acts only on the process output. An intermediate form is given by

$$u = k_p(\beta r - y) + k_i \int_0^\infty \big(r(\tau) - y(\tau)\big)\, d\tau + k_d\Big(\gamma \frac{dr}{dt} - \frac{dy}{dt}\Big), \qquad (10.14)$$

where the proportional and derivative actions act on fractions β and γ of the reference. Integral action has to act on the error to make sure that the error goes to zero in steady state. The closed loop systems obtained for different values of β and γ respond to load disturbances and measurement noise in the same way. The response to reference signals is different because it depends on the values of β and γ, which are called *reference weights* or *setpoint weights*. We illustrate the effect of setpoint weighting by an example.

Example 10.7 Cruise control with setpoint weighting
Consider the PI controller for the cruise control system derived in Example 10.3. Figure 10.12 shows the effect of setpoint weighting on the response of the system to a reference signal. With $\beta = 1$ (error feedback) there is an overshoot in velocity and the control signal (throttle) is initially close to the saturation limit. There is no overshoot with $\beta = 0$ and the control signal is much smaller, clearly a much better drive comfort. The frequency responses gives another view of the same effect. The parameter β is typically in the range 0–1, and γ is normally zero to avoid large transients in the control signal when the reference is changed. ∇

The controller given by equation (10.14) is a special case of the general controller structure having two degrees of freedom, which was discussed in Section 7.5.

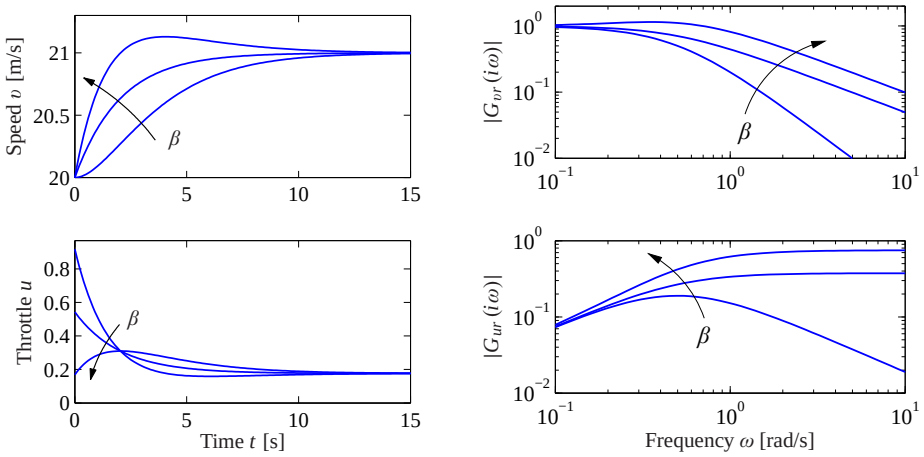
Implementation Based on Operational Amplifiers

PID controllers have been implemented in different technologies. Figure 10.13 shows how PI and PID controllers can be implemented by feedback around operational amplifiers.

To show that the circuit in Figure 10.13b is a PID controller we will use the approximate relation between the input voltage e and the output voltage u of the operational amplifier derived in Example 8.3,

$$u = -\frac{Z_2}{Z_1} e.$$

In this equation Z_1 is the impedance between the negative input of the amplifier and the input voltage e, and Z_2 is the impedance between the zero input of the amplifier



(a) Step response (b) Frequency responses

Figure 10.12: Time and frequency responses for PI cruise control with setpoint weighting. Step responses are shown in (a), and the gain curves of the frequency responses in (b). The controller gains are $k_p = 0.74$ and $k_i = 0.19$. The setpoint weights are $\beta = 0$, 0.5 and 1, and $\gamma = 0$.

and the output voltage u. The impedances are given by

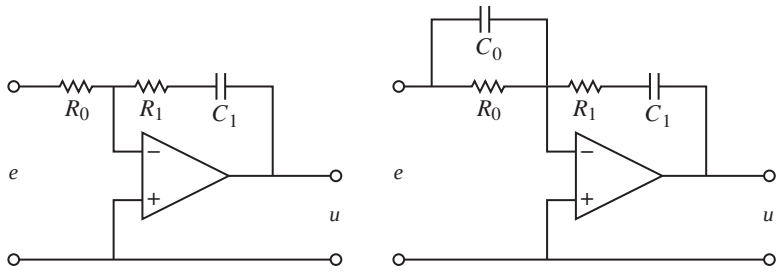
$$Z_1(s) = \frac{R_1}{1 + R_1 C_1 s}, \qquad Z_2(s) = R_2 + \frac{1}{C_2 s},$$

and we find the following relation between the input voltage e and the output voltage u:

$$u = -\frac{Z_2}{Z_1} e = -\frac{R_2}{R_1} \frac{(1 + R_1 C_1 s)(1 + R_2 C_2 s)}{R_2 C_2 s} e.$$

This is the input/output relation for a PID controller of the form (10.1) with parameters

$$k_p = \frac{R_2}{R_1}, \qquad T_i = R_2 C_1, \qquad T_d = R_1 C_1.$$



(a) PI controller (b) PID controller

Figure 10.13: Schematic diagrams for PI and PID controllers using op amps. The circuit in (a) uses a capacitor in the feedback path to store the integral of the error. The circuit in (b) adds a filter on the input to provide derivative action.

The corresponding results for a PI controller are obtained by setting $C_1 = 0$ (removing the capacitor).

Computer Implementation

In this section we briefly describe how a PID controller may be implemented using a computer. The computer typically operates periodically, with signals from the sensors sampled and converted to digital form by the A/D converter, and the control signal computed and then converted to analog form for the actuators. The sequence of operation is as follows:

1. Wait for clock interrupt
2. Read input from sensor
3. Compute control signal
4. Send output to the actuator
5. Update controller variables
6. Repeat

Notice that an output is sent to the actuators as soon as it is available. The time delay is minimized by making the calculations in step 3 as short as possible and performing all updates after the output is commanded. This simple way of reducing the latency is, unfortunately, seldom used in commercial systems.

As an illustration we consider the PID controller in Figure 10.11, which has a filtered derivative, setpoint weighting and protection against integral windup. The controller is a continuous-time dynamical system. To implement it using a computer, the continuous-time system has to be approximated by a discrete-time system.

A block diagram of a PID controller with anti-windup is shown in Figure 10.11. The signal v is the sum of the proportional, integral and derivative terms, and the controller output is $u = \text{sat}(v)$, where sat is the saturation function that models the actuator. The proportional term $k_p(\beta r - y)$ is implemented simply by replacing the continuous variables with their sampled versions. Hence

$$P(t_k) = k_p\left(\beta r(t_k) - y(t_k)\right), \tag{10.15}$$

where $\{t_k\}$ denotes the sampling instants, i.e., the times when the computer reads its input. We let h represent the sampling time, so that $t_{k+1} = t_k + h$. The integral term is obtained by approximating the integral with a sum,

$$I(t_{k+1}) = I(t_k) + k_i h\, e(t_k) + \frac{h}{T_t}\big(\text{sat}(v) - v\big), \tag{10.16}$$

where $T_t = h/k_t$ represents the anti-windup term. The filtered derivative term D is given by the differential equation

$$T_f \frac{dD}{dt} + D = -k_d y.$$

Approximating the derivative with a backward difference gives

$$T_f \frac{D(t_k) - D(t_{k-1})}{h} + D(t_k) = -k_d \frac{y(t_k) - y(t_{k-1})}{h},$$

which can be rewritten as

$$D(t_k) = \frac{T_f}{T_f + h} D(t_{k-1}) - \frac{k_d}{T_f + h} (y(t_k) - y(t_{k-1})) . \tag{10.17}$$

The advantage of using a backward difference is that the parameter $T_f/(T_f + h)$ is nonnegative and less than 1 for all $h > 0$, which guarantees that the difference equation is stable. Reorganizing equations (10.15)–(10.17), the PID controller can be described by the following pseudocode:

```
% Precompute controller coefficients
bi=ki*h
ad=Tf/(Tf+h)
bd=kd/(Tf+h)
br=h/Tt

% Control algorithm - main loop
while (running) {
  r=adin(ch1)               % read setpoint from ch1
  y=adin(ch2)               % read process variable from ch2
  P=kp*(b*r-y)              % compute proportional part
  D=ad*D-bd*(y-yold)        % update derivative part
  v=P+I+D                   % compute temporary output
  u=sat(v,ulow,uhigh)       % simulate actuator saturation
  daout(ch1)                % set analog output ch1
  I=I+bi*(r-y)+br*(u-v)     % update integral
  yold=y                    % update old process output
  sleep(h)                  % wait until next update interval
}
```

Precomputation of the coefficients `bi`, `ad`, `bd` and `br` saves computer time in the main loop. These calculations have to be done only when controller parameters are changed. The main loop is executed once every sampling period. The program has three states: `yold`, `I`, and `D`. One state variable can be eliminated at the cost of less readable code. The latency between reading the analog input and setting the analog output consists of four multiplications, four additions and evaluation of the `sat` function. All computations can be done using fixed-point calculations if necessary. Notice that the code computes the filtered derivative of the process output and that it has setpoint weighting and anti-windup protection.

10.6 Further Reading

The history of PID control is very rich and stretches back to the beginning of the foundation of control theory. Very readable treatments are given by Bennett [Ben86a, Ben86b] and Mindel [Min02]. The Ziegler–Nichols rules for tuning PID controllers, first presented in 1942 [ZN42], were developed based on extensive experiments with pneumatic simulators and Vannevar Bush's differential analyzer at MIT. An interesting view of the development of the Ziegler–Nichols rules is given in an interview with Ziegler [Bli90]. An industrial perspective on PID control is given in [Bia95],

[Shi96] and [YH91] and in the paper [DM02] cited in the beginning of this chapter. A comprehensive presentation of PID control is given in [ÅH05]. Interactive learning tools for PID control can be downloaded from http://www.calerga.com/contrib.

Exercises

10.1 (Ideal PID controllers) Consider the systems represented by the block diagrams in Figure 10.1. Assume that the process has the transfer function $P(s) = b/(s+a)$ and show that the transfer functions from r to y are

$$(a) \quad G_{yr}(s) = \frac{bk_d s^2 + bk_p s + bk_i}{(1+bk_d)s^2 + (a+bk_d)s + bk_i},$$

$$(b) \quad G_{yr}(s) = \frac{bk_i}{(1+bk_d)s^2 + (a+bk_d)s + bk_i}.$$

Pick some parameters and compare the step responses of the systems.

10.2 Consider a second-order process with the transfer function

$$P(s) = \frac{b}{s^2 + a_1 s + a_2}.$$

The closed loop system with a PI controller is a third-order system. Show that it is possible to position the closed loop poles as long as the sum of the poles is $-a_1$. Give equations for the parameters that give the closed loop characteristic polynomial

$$(s+\alpha_0)(s^2 + 2\zeta_0\omega_0 s + \omega_0^2).$$

10.3 Consider a system with the transfer function $P(s) = (s+1)^{-2}$. Find an integral controller that gives a closed loop pole at $s = -a$ and determine the value of a that maximizes the integral gain. Determine the other poles of the system and judge if the pole can be considered dominant. Compare with the value of the integral gain given by equation (10.6).

10.4 (Ziegler–Nichols tuning) Consider a system with transfer function $P(s) = e^{-s}/s$. Determine the parameters of P, PI and PID controllers using Ziegler–Nichols step and frequency response methods. Compare the parameter values obtained by the different rules and discuss the results.

10.5 (Vehicle steering) Design a proportional-integral controller for the vehicle steering system that gives the closed loop characteristic polynomial

$$s^3 + 2\omega_0 s^2 + 2\omega_0 s + \omega_0^3.$$

10.6 (Congestion control) A simplified flow model for TCP transmission is derived in [HMTG00, LPD02]. The linearized dynamics are modeled by the transfer function

$$G_{qp}(s) = \frac{b}{(s+a_1)(s+a_2)} e^{-s\tau_e},$$

which describes the dynamics relating the expected queue length q to the expected packet drop p. The parameters are given by $a_1 = 2N^2/(c\tau_e^2)$, $a_2 = 1/\tau_e$ and $b = c^2/(2N)$. The parameter c is the bottleneck capacity, N is the number of sources feeding the link and τ_e is the round-trip delay time. Use the parameter values $N = 75$ sources, $C = 1250$ packets/s and $\tau_e = 0.15$ and find the parameters of a PI controller using one of the Ziegler–Nichols rules and the corresponding improved rule. Simulate the responses of the closed loop systems obtained with the PI controllers.

10.7 (Motor drive) Consider the model of the motor drive in Exercise 2.10. Develop an approximative second-order model of the system and use it to design an ideal PD controller which gives a closed loop system with eigenvalues in $\zeta\omega_0 \pm i\omega_0\sqrt{1-\zeta^2}$. Add low-pass filtering as shown in equation (10.13) and explore how large ω_0 can be made while maintaining a good stability margin. Simulate the closed loop system with the chosen controller and compare the results with the controller based on state feedback in Exercise 6.11.

10.8 Consider the system in Exercise 10.7 investigate what happens if the second-order filtering of the derivative is replace by a first-order filter.

10.9 (Tuning rules) Apply the Ziegler–Nichols and the modified tuning rules to design PI controllers for systems with the transfer functions

$$P_1 = \frac{e^{-s}}{s}, \qquad P_2 = \frac{e^{-s}}{s+1}, \qquad P_s = e^{-s}.$$

Compute the stability margins and explore any patterns.

10.10 (Windup and anti-windup) Consider a PI controller of the form $C(s) = 1+1/s$ for a process with input that saturates when $|u| > 1$, and whose linear dynamics are given by the transfer function $P(s) = 1/s$. Simulate the response of the system to step changes in the reference signal. Repeat the simulation when the windup protection scheme in Figure 10.11 is used.

10.11 (Windup protection by conditional integration) Many methods have been proposed to avoid integrator windup. One method called *conditional integration* is to update the integral only when the error is sufficiently small. To illustrate this method we consider a system with PI control described by

$$\frac{dx_1}{dt} = u, \qquad u = \text{sat}(k_p e + k_i x_2, -u_0, u_0), \qquad \frac{dx_2}{dt} = \begin{cases} e & \text{if } |e| < 1 \\ 0 & \text{if } |e| \geq e_0, \end{cases}$$

where $e = r - x$. Plot the phase portrait of the system for the parameter values $k_p = 1$, $k_i = 1$, $u_0 = 1$ and $e_0 = 1$ and discuss the properties of the system. The example illustrates the difficulties of introducing ad hoc nonlinearities without careful analysis.